

07 The Transport Layer

07.05 Transmission Control Protocol (TCP)

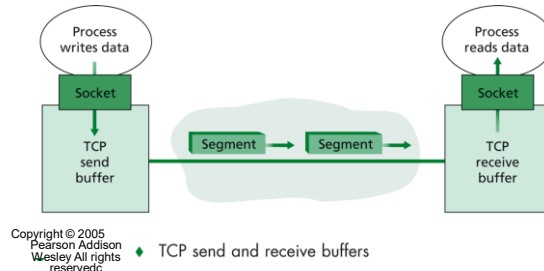
Copyright note: all figures and tables reported, if not specified, are taken from: A. Tanenbaum: *Computer Networks*, 5th ed. Pearson Prentice Hall
(Others are expressly specified)

Transport Control Protocol

- Connection oriented (3 way handshake)
- Reliable service
 - Correct segments are acknowledged
 - Possible errors are managed by re-transmissions (timeout)
- Ordered sequence
- Pipelining: flow control, congestion control use TCP window
- Flow control: sender cannot send more than receiver can receive
- Buffering on sender and receiver side
- Full duplex communication
- TCP must provide the reliability that IP doesn't provide

The TCP Service Model

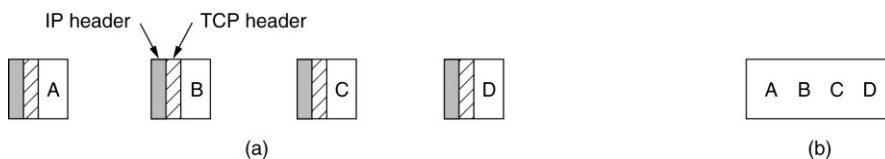
- TCP service is obtained creating final points (sockets)
- IP host address, 16 bit port number
- Port number < 1024: *well known ports*
- It could be possible to associate a daemon to a port when host boots (expensive)
- Unix has 1 daemon `inetd` (Internet daemon) waiting for input connection
- When the connection is set up `inetd` creates a new process with proper daemon
- `inetd` knows ports from a configuration file



Port	Protocol	Use
21	FTP	File transfer
23	Telnet	Remote login
25	SMTP	E-mail
69	TFTP	Trivial File Transfer Protocol
79	Finger	Lookup info about a user
80	HTTP	World Wide Web
110	POP-3	Remote e-mail access
119	NNTP	USENET news

The TCP Service Model

- A TCP connection is a data flow
- Message boundaries are not preserved
- When the application passes data to TCP, data can be sent or buffered (more bytes sent together)
- The application may force data sending (e.g. telnet after return) using the PUSH flag
- TCP handles urgent data (e.g. ctrl+c to interrupt a remote execution) using the URGENT flag

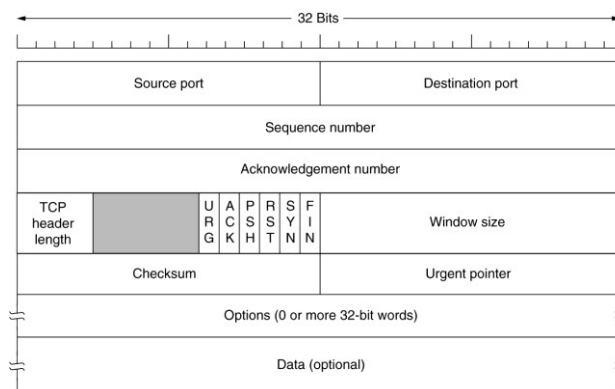


The TCP Service Model

- Each byte in the TCP connection has a 32 bit sequence number
- TCP entities exchange segments: 20 byte fixed header (+ options) + data
- TCP software decides the size of segments
 - It can put in one segments data from more writings
 - It can split one writing in more segments
 - The MSS (Maximum Segment Size) is generally chosen based on the data link MTU, so that a TCP segment once framed into an IP packet fits MTU
- TCP entities use sliding window protocol
 - Sender sends a segment and starts a timer
 - Receiver answers (possible piggybacking) ACK = next waited seq.nr
 - In case of timeout → re-transmission

The TCP Segment Header

- TCP Header
- Segments without data are usable (ACK and control messages)

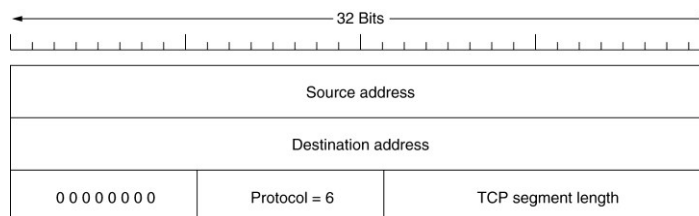


The TCP Segment Header

- Source port and destination port: they identify the port used in the connection (multiplexing/demultiplexing). Port+IP address locate end points. Source and destination end points identify a connection
- Sequence number, ACK number: are used to implement reliable data transfer. ACK number refers to the next waited byte
- TCP header length: how many 32 bit words are contained in TCP header (usually option is empty so header = 20 byte)
- URG: set when Urgent Pointer is used
- ACK: set to 1 to denote that acknowledge number is valid
- PSH: push data
- RST: used to refuse a connection or a non-valid segment
- SYN: used in connection set up
 - CONNECTION REQUEST: SYN=1, ACK=0
 - CONNECTION ACCEPTED: SYN=1, ACK=1
- FIN: used to release connection
- Window size: flow control (go back N or selective repeat (RFC 1106))
- Checksum: 16 bit word of TCP segment are added (header+data+pseudoheader) and then complemented to 1
- Options: used for MSS negotiation (default: 536 byte payload) or window scaling factor in high speed networks

The TCP Segment Header

- The pseudoheader included in the TCP checksum.
- It includes 32 bit IP addresses
- It can be used for checking packets not correctly delivered, but including IP addresses violates the hierarchy levels (IP addresses belong to IP layer, not to TCP)



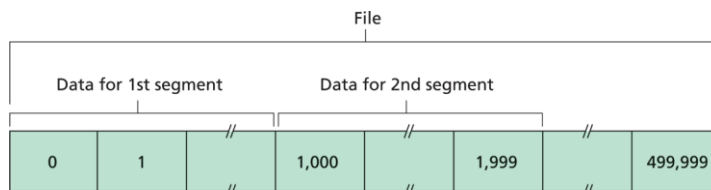
The pseudoheader included in the TCP checksum.

The TCP Segment Header

- Options field can be used for window scaling
- In case of lines with high bandwidth or high delay a 64 KB window can often be a problem
- Sender could be inactive for a long time
- RFC 1323 allows sender and receiver to negotiate a window scale allowing a window up to 2^{30} byte
- Another option proposed by RFC 1106 is the use of selective repeat instead of go back n

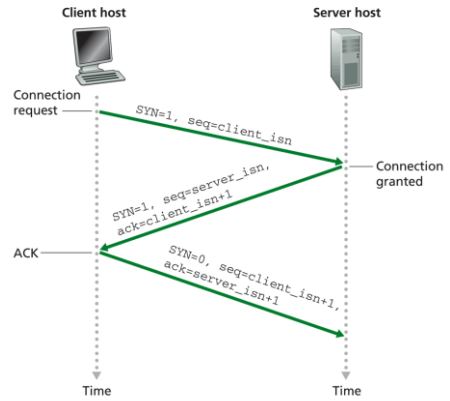
The TCP Segment Header

- How are sequence numbers in TCP segments handled?
- Suppose a data flow that consists of a 500000 byte file
- Suppose MSS = 1000 byte
- TCP could build 500 segments (1000 byte each)
- 1st segment → 0
- 2nd segment → 1000, etc.



TCP Connection Establishment

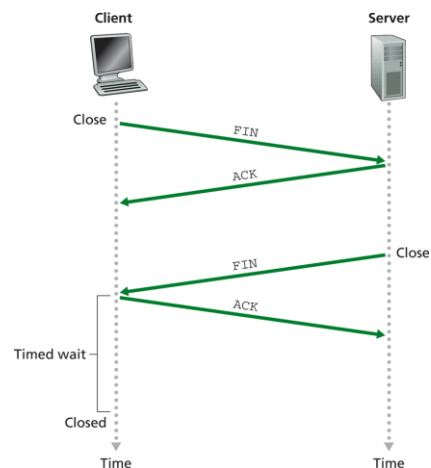
- Server waits passively for an incoming connection: LISTEN, ACCEPT
- Client specifies IP address and port and actively contact server:
CONNECT → a TCP segment with SYN=1, ACK=0
- Server TCP entity checks if a process have called LISTEN on that port (destination port of SYN segment):
 - If no → send a segment with RST=1 to refuse connection
 - If yes the connection can be accepted or not → if accepted buffers and variables are allocated and a "connection granted message" is sent back (SYNACK segment)
 - When SYNACK is received, client allocates buffers and variables and acknowledges SYNACK
- Initial sequence number for connection is randomly chosen (usually using a clock-based scheme)



Copyright © 2005 Pearson Addison Wesley All rights reserved
♦ TCP three-way handshake: segment exchange

TCP Connection Release

- 4 TCP segments are usually used (they can be 3 if 1st ACK and 2nd FIN are sent in the same segment)
- Timers are used : if the answer to FIN doesn't arrive after a given time, the sender of FIN releases the connection



Copyright © 2005 Pearson Addison Wesley All rights reserved
♦ Closing a TCP connection

TCP Connection Management Modeling

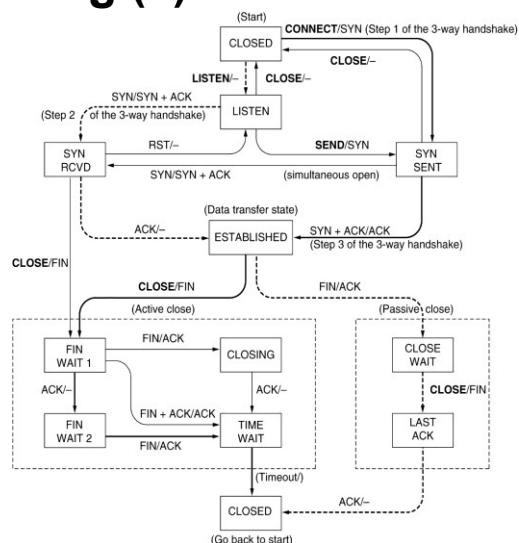
- The states used in the TCP connection management finite state machine.

State	Description
CLOSED	No connection is active or pending
LISTEN	The server is waiting for an incoming call
SYN RCVD	A connection request has arrived; wait for ACK
SYN SENT	The application has started to open a connection
ESTABLISHED	The normal data transfer state
FIN WAIT 1	The application has said it is finished
FIN WAIT 2	The other side has agreed to release
TIMED WAIT	Wait for all packets to die off
CLOSING	Both sides have tried to close simultaneously
CLOSE WAIT	The other side has initiated a release
LAST ACK	Wait for all packets to die off

The states used in the TCP connection management finite state machine.

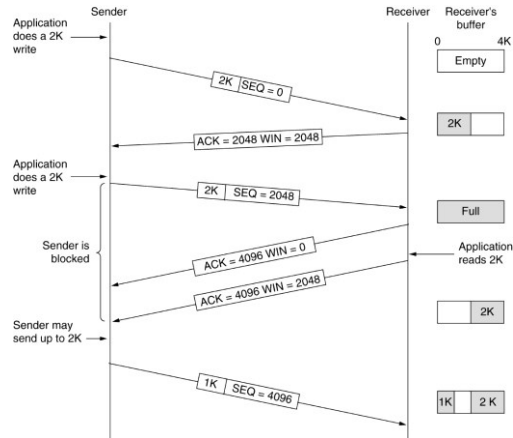
TCP Connection Management Modeling (2)

TCP connection management finite state machine. The heavy solid line is the normal path for a client. The heavy dashed line is the normal path for a server. The light lines are unusual events. Each transition is labeled by the event causing it and the action resulting from it, separated by a slash.



TCP Transmission Policy

- Window management in TCP
- When the window is 0, the sender cannot send data, with 2 exceptions:
 - Urgent data (e.g. allow to interrupt the remote process)
 - 1 byte segment to trigger (*next waited byte, window size*) from receiver, to avoid deadlocks

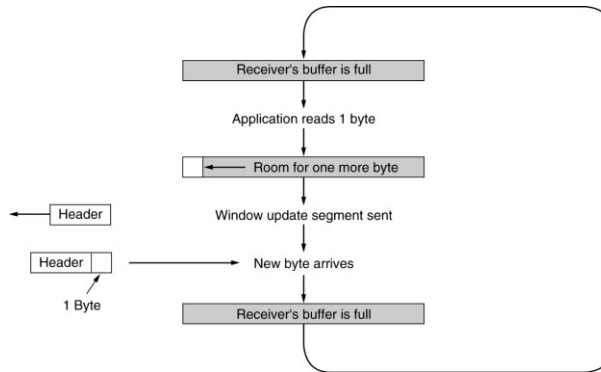


TCP Transmission Policy

- Consider an interactive telnet session (1 char is sent):
 - Sender: TCP segment + IP header (20+20+1=41 byte)
 - Receiver:
 - ACK (40 byte)
 - When the editor reads char, TCP sends the update of window (40 byte)
 - The editor process the character and sends back an echo (41 byte)
 - tot = 162 byte, 4 segments, 1 char
- 1st solution used in many TCP implementations:
 - Delay ACKs and window update (max 500 msec)
 - Receiver application may use some data (no small window update)
 - Receiver application may have some data to send (ACK and update piggybacked)
- Nagle algorithm:
 - If application writes data 1 byte at the time:
 - Send 1st byte
 - Put the rest into buffer until the ACK comes back
 - Send all chars in buffer in 1 segment
 - If buffered bytes are > MSS or 50% sender window send segment

TCP Transmission Policy

- The “*silly window syndrome*” is a problem that can degrade performance in TCP transmission
- We have it if data are passed to TCP sender entity in large blocks but the receiver application read data 1 byte/read

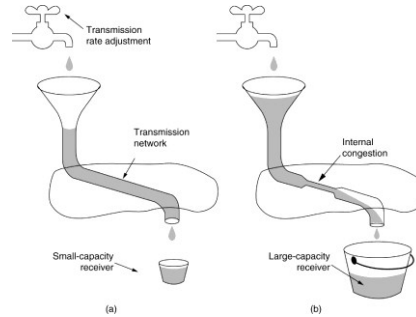


TCP Transmission Policy

- Solution to s.w.s.:
 - Clark solution: the receiver is not allowed to send an update of the window for 1 byte
 - The receiver can update send the update only if it has enough space to receive an MSS or when its buffer is half-empty
 - Sender could help by not sending small segments, waiting enough window space x a segment, or a segment x $\frac{1}{2}$ receiver buffer
- Nagle algorithm and Clark solutions are complementary solutions to s.w.s

TCP Congestion Control

- Network layer tries to solve congestions
- TCP is an effective approach
- TCP algorithms assume timeouts are due to congestion
- Bottleneck can shift from receiver capacity to network capacity



(a) A fast network feeding a low capacity receiver.
(b) A slow network feeding a high-capacity receiver.

TCP Congestion Control

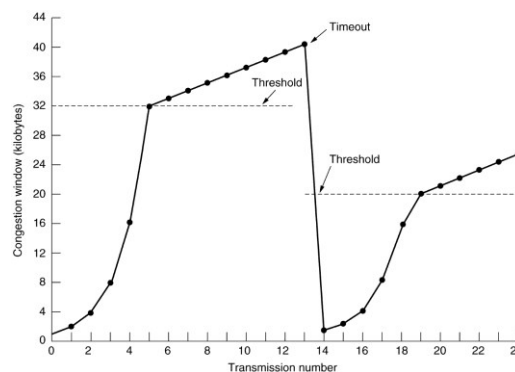
- Network and receiver capacities are separately managed
- Sender keeps two windows: the receiver w. (rw) and the congestion w. (cw) $\rightarrow$ the smallest is used
- After connection is established, the cw is set to MSS
- If segment is acknowledged before timeout, $cw += MSS$ and 2 segments are sent, etc.
- When cw reaches the size of n segments (each ack'ed in time) $\rightarrow cw += n * \text{sizeof}(\text{segment})$
- Exponential growth up to timeout or rw saturation (it is known as slow start algorithm, but is not slow!)
- $1024 \rightarrow 2048 \rightarrow 4096 \rightarrow 8192$ $cw=4096$

TCP Congestion Control

- The algorithm also uses a threshold (th)
- th = 64 KB at start
- If timeout $\rightarrow$ th = $\frac{1}{2}$ cw, cw = MSS
- So slow start is used to see what the network is able to do, then when this reached:
 - Successful transmission: $cw += \text{sizeof}(\text{segment})$
 $\rightarrow$ cw grow linearly, no more exponentially
- See figure on next slide

TCP Congestion Control

- An example of the Internet congestion algorithm
- MSS=1024 byte, th = 64 KB
- Timeout $\rightarrow$ th=32 KB, cw=1KB for trasmission 0



TCP Congestion Control

- end-end control (no network assistance)
- sender limits transmission:
 $\text{LastByteSent} - \text{LastByteAcked} \leq \text{CongWin}$
- Roughly,

- **CongWin** is dynamic, function of perceived network congestion

$$\text{rate} = \frac{\text{CongWin}}{\text{RTT}} \cdot \text{Bytes/sec}$$

How does sender perceive congestion?

- loss event = timeout or 3 duplicate acks
- TCP sender reduces rate (**CongWin**) after loss event

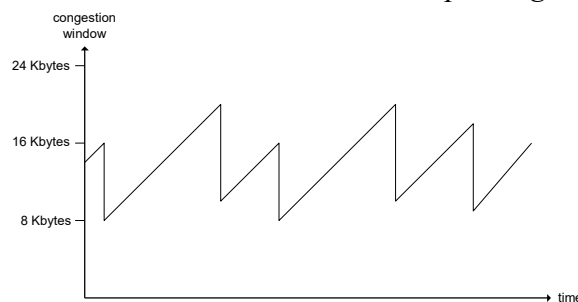
three mechanisms:

- AIMD
- slow start
- conservative after timeout events

TCP AIMD

multiplicative decrease:
cut **CongWin** in half
after loss event

additive increase: increase
CongWin by 1 MSS every
RTT in the absence of loss
events: *probing*



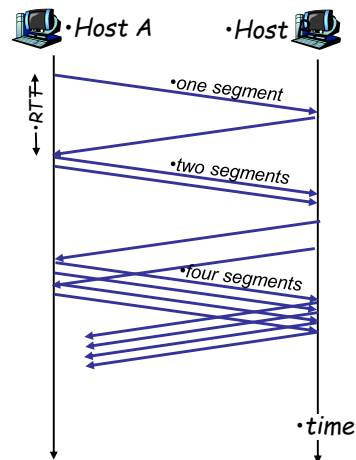
Long-lived TCP connection

TCP Slow Start

- When connection begins, **CongWin** = 1 MSS
 - Example: MSS = 500 bytes & RTT = 200 msec
 - initial rate = 20 kbps
- available bandwidth may be $\gg$ MSS/RTT
 - desirable to quickly ramp up to respectable rate
- *When connection begins, increase rate exponentially fast until first loss event*

TCP Slow Start (more)

- When connection begins, increase rate exponentially until first loss event:
 - double **CongWin** every RTT
 - done by incrementing **CongWin** for every ACK received
- **Summary:** initial rate is slow but ramps up exponentially fast



Refinement

- After 3 dup ACKs:
 - **CongWin** is cut in half
 - window then grows linearly
- But after timeout event:
 - **CongWin** instead set to 1 MSS;
 - window then grows exponentially
 - to a threshold, then grows linearly

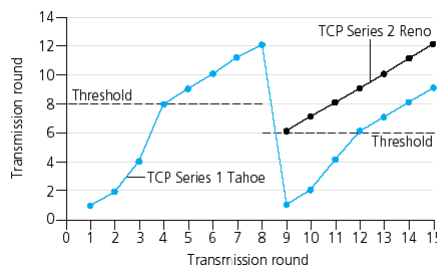
•Philosophy:

- 3 dup ACKs indicates network capable of delivering some segments
- timeout before 3 dup ACKs is "more alarming"

Refinement (more)

Q: When should the exponential increase switch to linear?

A: When **CongWin** gets to 1/2 of its value before timeout.



Implementation:

- Variable Threshold
- At loss event, Threshold is set to 1/2 of CongWin just before loss event

Summary: TCP Congestion Control

- When **CongWin** is below **Threshold**, sender in **slow-start** phase, window grows exponentially.
- When **CongWin** is above **Threshold**, sender is in **congestion-avoidance** phase, window grows linearly.
- When a **triple duplicate ACK** occurs, **Threshold** set to **CongWin/2** and **CongWin** set to **Threshold**.
- When **timeout** occurs, **Threshold** set to **CongWin/2** and **CongWin** is set to 1 MSS.

TCP sender congestion control

Event	State	TCP Sender Action	Commentary
ACK receipt for previously unacked data	Slow Start (SS)	CongWin = CongWin + MSS, If (CongWin > Threshold) set state to "Congestion Avoidance"	Resulting in a doubling of CongWin every RTT
ACK receipt for previously unacked data	Congestion Avoidance (CA)	CongWin = CongWin+MSS * (MSS/CongWin)	Additive increase, resulting in increase of CongWin by 1 MSS every RTT
Loss event detected by triple duplicate ACK	SS or CA	Threshold = CongWin/2, CongWin = Threshold, Set state to "Congestion Avoidance"	Fast recovery, implementing multiplicative decrease. CongWin will not drop below 1 MSS.
Timeout	SS or CA	Threshold = CongWin/2, CongWin = 1 MSS, Set state to "Slow Start"	Enter slow start
Duplicate ACK	SS or CA	Increment duplicate ACK count for segment being acked	CongWin and Threshold not changed

TCP throughput

- What's the average throughput of TCP as a function of window size and RTT?
 - Ignore slow start
- Let W be the window size when loss occurs.
- When window is W , throughput is W/RTT
- Just after loss, window drops to $W/2$, throughput to $W/2RTT$.
- Average throughput: $.75 W/RTT$

TCP Futures

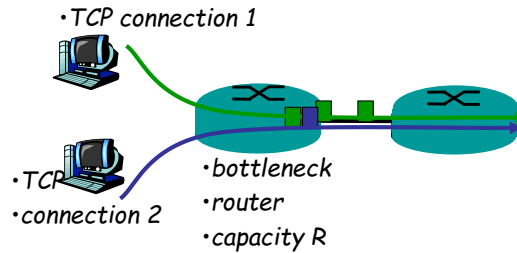
- Example: 1500 byte segments, 100ms RTT, want 10 Gbps throughput
- Requires window size $W = 83,333$ in-flight segments
- Throughput in terms of loss rate:

$$\frac{1.22 \cdot MSS}{RTT \sqrt{L}}$$

- $\rightarrow L = 2 \cdot 10^{-10}$
- New versions of TCP for high-speed needed!

TCP Fairness

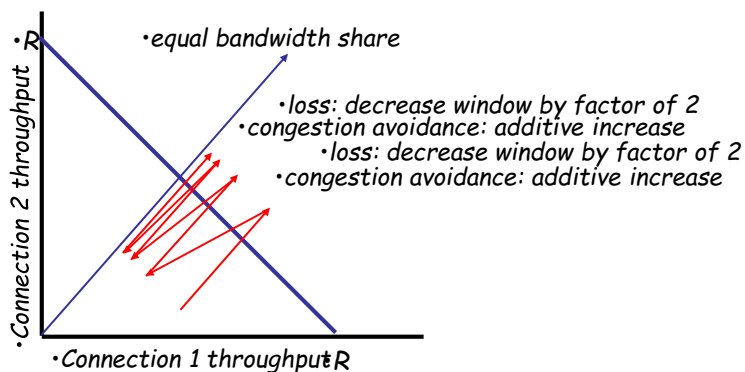
Fairness goal: if K TCP sessions share same bottleneck link of bandwidth R , each should have average rate of R/K



Why is TCP fair?

Two competing sessions:

- Additive increase gives slope of 1, as throughput increases
- multiplicative decrease decreases throughput proportionally



Fairness (more)

Fairness and UDP

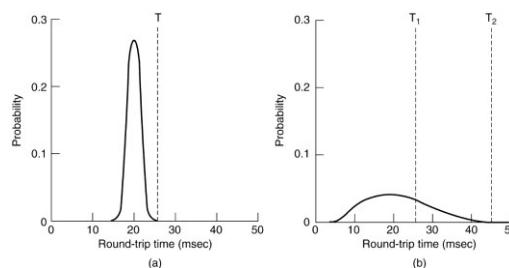
- Multimedia apps often do not use TCP
 - do not want rate throttled by congestion control
- Instead use UDP:
 - pump audio/video at constant rate, tolerate packet loss
- Research area: TCP friendly

Fairness and parallel TCP connections

- nothing prevents app from opening parallel connections between 2 hosts.
- Web browsers do this
- Example: link of rate R supporting 9 connections;
 - new app asks for 1 TCP, gets rate $R/10$
 - new app asks for 11 TCPs, gets $R/2$!

TCP Timer Management

- How long should be the timeout?
- T_1 : we will have unnecessary re-transmissions that will congest the Internet
- T_2 : Performances will degrade due to long delay each time a packet is lost
- Moreover average and variance can sensibly change
- Solution: a dynamic algorithm (Jacobson, 1988)



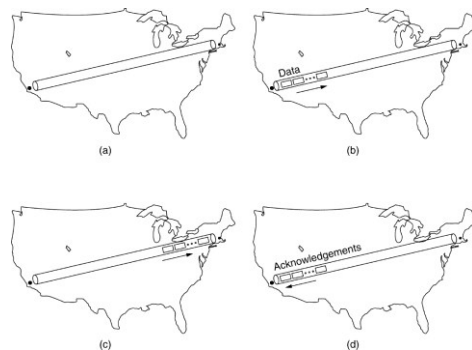
- (a) Probability density of ACK arrival times in the data link layer. (b) Probability density of ACK arrival times for TCP

TCP Timer Management

- Solution: a dynamic algorithm (Jacobson, 1988)
- TCP keeps for each connection a variable, RTT
- RTT represents an estimate of round-trip-time
- If ACK comes back before timeout (at time M), RTT is updated:
 - $RTT = \alpha RTT + (1 - \alpha)M$
 - α weights the old value (is usually 7/8)
 - It also computed an estimate (D) of the standard deviation:
 - $D = \alpha D + (1 - \alpha)|RTT - M|$
- Most TCP implementation compute Timeout as:
 - Timeout = RTT + 4D
- Other timer used in TCP:
 - Persistence timer: used to avoid deadlock
 - Keepalive timer: used for connections inactive for long time to check if the other side is connected
 - Timer used in TIMED WAIT state during connection closing

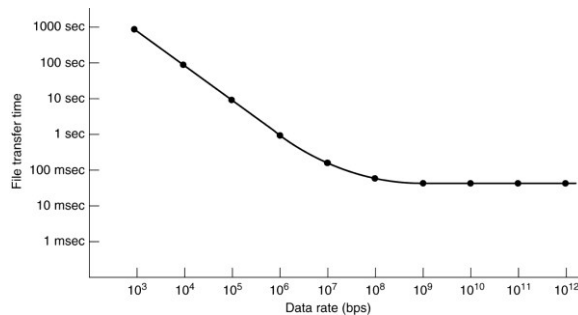
Performance Problems in Computer Networks

- The state of transmitting one megabit from San Diego to Boston
- (a) At $t = 0$, (b) After 500 μsec , (c) After 20 msec, (d) after 40 msec.
- Line is used 0.5/40 (1.25%)
- Bandwidth $\times$ delay (= 40×10^6 in this example)
- Hence the receiver window should be this value



Protocols for Gigabit Networks

- The advent of gigabit networks has impacted several factors
- 32 bit seq.nr.
 - the cycle time for seq.nr.
 - 1 week on 56kbps lines
 - 57 min on 10 Mbps Ethernet
 - 34 sec on 1 Gbps (the min. lifetime of packets on the Internet can be assumed to be 120 sec)
- CPU time have increased less than communication speed
 - Less time for processing protocols
- Gigabit networks are limited by delay and not by bandwidth



Time to transfer and acknowledge a 1-megabit file over a 4000-km line.

WebRTC

- WebRTC is a combination of standards, protocols, and JavaScript APIs that enables real-time communications (RTC).
- Users connecting via Chrome, Firefox, or Safari can communicate directly through their browsers
- WebRTC started as a Google open-source project aimed at giving browsers the ability to support real-time voice and video communication without any plug-ins at 2009
- IETF and W3C get involved at 2011

How does WebRTC work?

- WebRTC uses three HTML5 APIs to capture, encode, and transmit live streams between one another's browsers, enabling two-way communication (it is peer-to-peer technology).
- It eliminates the need for any intermediary web servers or additional equipment (Hw or Sw) during these exchanges.
- WebRTC can be played back on any HTML5 player.

WebRTC Benefits

- **Inherently Low Latency.** WebRTC it's important for the delivery speed. WebRTC offers one of the quickest method for transporting video across the internet (At sub-500-milliseconds latency end to end).
- **Platform and Device Independence.** All major browsers and devices support WebRTC. Since WebRTC makes use of HTML5 APIs, it allows you to utilize many of the features built into the HTML5 programming language through a lightweight, embedded framework..

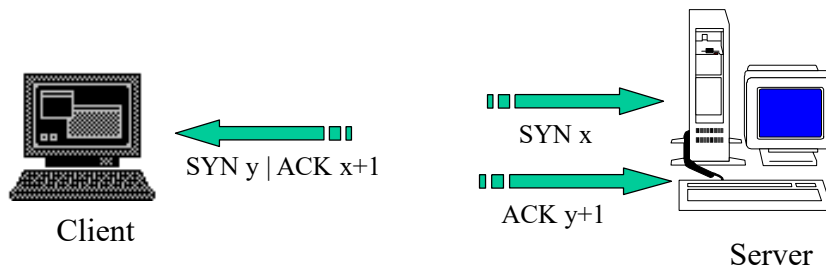
WebRTC Benefits

- **Open Source and Standardized.** The open-source framework is standardized by the IETF and W3C, thereby eliminating any interoperability challenges that come with proprietary streaming technologies and with aid of thousands of software developers working on it in concert, standardizing conferencing protocols and making.
- **Adapts to Network Conditions.** WebRTC ensures reliable publishing over poor network conditions with adaptive network encoding. That's because it supports a capability called 'simulcasting' (the client generates multiple streams at varying bitrates and quality). Unlike adaptive bitrate streaming, where the stream dynamically adjusts during playback, this takes place at the publishing side and provides multiple encodings rather than the ability to adapt bitrate mid-stream.

WebRTC Limitations

- **Scalability.** WebRTC simply wasn't designed with scalability in mind. The bandwidth-intensive configuration requires each participating browser to connect with each other via a peer connection.
- **Broadcast Quality.** A common misconception is that WebRTC lacks in quality due to bitrate limitations. While browser-based contribution is inherently connectivity- and camera-dependent on the resolution front, high-bitrate encoding is still possible (real-time delivery can impact quality).

TCP Security problems



Issues?

- Server needs to keep waiting for ACK $y+1$
- Server recognizes Client based on IP address/port and $y+1$

TCP SYN Flooding

- Exploit state allocated at server after initial SYN packet
- Send a SYN and don't reply with ACK
- Server will wait for 511 seconds for ACK
- Finite queue size for incomplete connections (1024)
- Once the queue is full it doesn't accept requests

TCP Session Hijack

- When is a TCP packet valid?
 - Address/Port/Sequence Number in window
- How to get sequence number?
 - Sniff traffic
 - Guess it
 - Many earlier systems had predictable ISN
- Inject arbitrary data to the connection
- The server accepts it and increases the expected seq-number for the next one.
- As soon the next packet from the real client arrives the server drops it as outdated, so the client is desynchronized and loses the connection

TCP Session Reset

- The attacker sends packets with RST Flag ON to both A and B or any one of the host.
- Since both A and B do not know that an attacker has sent these packets so they treat these packets normally.
- Since they are reset packets so connection between A and B is terminated.
- So we can see that TCP reset attacks are aimed to terminate a valid TCP connection between two hosts.